Advanced Priority Queues, In-Place Heap Sorting, Dynamic Data Stream Quantiles, and Optimal Interval Scheduling: A Formal Algorithmic Analysis

Abstract

This paper presents a formal, unified algorithmic analysis of advanced heap operations and greedy interval scheduling derived from classroom instruction on June 1, 2026. First, to ensure complete mathematical accessibility, we provide a comprehensive reference guide explaining every mathematical symbol, floor/ceiling function, binary logarithm, summation operator, and set notation used in asymptotic analysis. Second, we formalize in-place Heap Sort using Floyd's O(N) build-heap algorithm, analyze its total O(N log N) time and O(1) auxiliary space complexity, and present a formal proof of its inherent structural instability. Third, we establish the dynamic dual-heap architecture (Max-Heap + Min-Heap) for computing continuous data stream medians in O(log N) insertion and O(1) query time. Fourth, we examine the Interval Scheduling problem (Maximum Jobs Completed), analyze counterexamples to intuitive greedy heuristics, and present a rigorous inductive "Greedy Stays Ahead" proof of global optimality for the Earliest Finish Time (EFT) algorithm. Complete, compilable production-grade Java reference implementations are provided for all discussed algorithms.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Mathematical Symbols, Formal Notation, and Accessible Reference Guide

To ensure complete clarity and accessibility for readers of all mathematical backgrounds, this section provides an explicit, comprehensive reference guide to every mathematical symbol, set notation, and asymptotic function utilized throughout this research paper. Computer science literature often uses concise notation that can obscure underlying physical or algorithmic intuition. Below, each symbol is defined with its formal mathematical meaning, plain-language translation, and specific algorithmic application.

1.1 Asymptotic Bounds and Complexity Notations

Asymptotic notation characterizes the growth rate of execution time or memory space as the input size N approaches infinity.

- Big-O Notation ($\mathcal{O}$ or O): Upper Bound
  - Formal Definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $f(N) \leq c \cdot g(N)$ for all $N \geq N_0$.
  - Plain Explanation: Represents the worst-case scenario or maximum resource limit. It guarantees that the execution time will not exceed a multiple of $g(N)$.
  - Example in Paper: Inserting an element into a binary heap of size N requires $O(\log N)$ comparisons in the worst case.
- Big-Omega Notation (Ω): Lower Bound
  - Formal Definition: $f(N) = \Omega(g(N))$ if there exist positive constants c and N_0 such that $f(N) \geq c \cdot g(N)$ for all $N \geq N_0$.
  - Plain Explanation: Represents the best-case scenario or absolute minimum work required by any algorithm solving a given problem.
  - Example in Paper: Any comparison-based sorting algorithm requires $\Omega(N \log N)$ comparisons to sort N arbitrary elements.
- Big-Theta Notation (Θ): Tight Bound
  - Formal Definition: $f(N) = \Theta(g(N))$ if $f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$.
  - Plain Explanation: Represents an exact asymptotic bound. The execution time grows at precisely the same rate as $g(N)$ within constant factors.
  - Example in Paper: Building a heap from an arbitrary array using Floyd's bottom-up algorithm takes $\Theta(N)$ time.

1.2 Structural Functions and Operators

- Floor Function ($\lfloor x \rfloor$):
  - Definition: Maps a real number x to the largest integer less than or equal to x (i.e., rounding down).
  - Example: $\lfloor 3.7 \rfloor = 3$, $\lfloor 4.0 \rfloor = 4$.
  - Algorithmic Use: Used to calculate array index of a parent node in zero-indexed binary trees: $\text{parent}(i) = \lfloor \frac{i-1}{2} \rfloor$.
- Ceiling Function ($\lceil x \rceil$):
  - Definition: Maps a real number x to the smallest integer greater than or equal to x (i.e., rounding up).
  - Example: $\lceil 3.1 \rceil = 4$, $\lceil 4.0 \rceil = 4$.
  - Algorithmic Use: Used to bound the number of nodes at level l in a binary tree: $n_l = \lceil \frac{N}{2^{l+1}} \rceil$.
- Binary Logarithm ($\log_2 N$ or $\log N$):
  - Definition: The exponent to which the base 2 must be raised to produce N. Equivalently, the number of times N can be divided by 2 until reaching 1.
  - Example: $\log_2(8) = 3$ because $2^3 = 8$.
  - Algorithmic Use: Dictates the maximum height $H = \lfloor \log_2 N \rfloor$ of a complete binary tree containing N nodes.
- Summation Operator ($\sum$):
  - Definition: Computes the sum of a sequence of terms: $\sum_{i=a}^{b} f(i) = f(a) + f(a+1) + \cdots + f(b)$.
  - Example: $\sum_{i=1}^{3} i^2 = 1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14$.
  - Algorithmic Use: Summing time contributions across tree levels during heap construction.
- Set Membership and Quantifiers ($\in, \forall, \exists$):
  - $\in$: "Is an element of". Example: $i \in \{0, 1, \ldots, N-1\}$ means index i is within array bounds.
  - $\forall$: "For all" or "For every". Example: $\forall x \in H_1$, $x \leq y$ means every item in set H_1 is less than or equal to y.
  - $\exists$: "There exists". Example: $\exists k$ such that $e_k \leq s_j$.
  - $\implies$: "Implies" or "Logically leads to".

TABLE 1
Quick Reference Guide to Mathematical Notation Used in the Paper

Symbol	Name / Meaning	Plain-Language Explanation
$O(g(N))$	Big-O (Worst-case)	Upper bound on execution time (won't be worse than this).
$\Omega(g(N))$	Big-Omega (Best-case)	Lower bound on execution time (will take at least this long).
$\Theta(g(N))$	Big-Theta (Tight bound)	Exact growth rate of execution time.
$\lfloor x \rfloor$	Floor function	Round down to nearest whole integer.
$\lceil x \rceil$	Ceiling function	Round up to nearest whole integer.
$\log_2 N$	Logarithm base 2	Tree height / number of times N can be halved.
$\sum_{i=a}^{b}$	Summation	Add together values from $i = a$ up to $i = b$.
$\in$	Set element	Belongs to / is inside a group.
$\forall$	Universal quantifier	"For every single item in the set".
$\exists$	Existential quantifier	"There exists at least one item".
$\max(A)/\min(A)$	Extremum functions	Maximum value / Minimum value in set A.
$H = \lfloor \log_2 N \rfloor$	Tree Height	Total vertical levels in a complete binary tree.

2 Fundamental Binary Heap Architecture & In-Place Heap Sort

Binary heaps serve as the backbone for priority queue abstract data types (ADTs) and in-place sorting algorithms. Derived from classroom instruction on June 1, 2026, this section formalizes the binary heap structural invariants, compares multi-language standard library implementations, and analyzes the in-place Heap Sort algorithm alongside its fundamental stability properties.

2.1 Structural Invariants and Implicit Array Mapping

A Binary Heap is a complete binary tree mapped into a contiguous one-dimensional array without allocating explicit left and right child pointers or node objects.

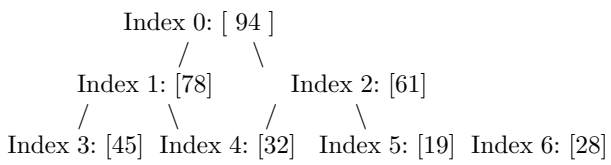
Definition 1 (Complete Binary Tree (CBT)). A binary tree T of height H is complete if all levels $0, 1, \ldots, H-1$ are fully populated with nodes, and the final level H is populated strictly from left to right without structural gaps.

Definition 2 (Heap Ordering Invariants).
- Max-Heap Invariant: For every non-root node at index i, $A[\text{parent}(i)] \geq A[i]$. The root $A[0]$ stores the maximum element.

- Min-Heap Invariant: For every non-root node at index i, $A[\text{parent}(i)] \leq A[i]$. The root $A[0]$ stores the minimum element.

For any zero-indexed position $i \in \{0, 1, \ldots, N-1\}$, the positional index mapping formulas are defined as:

$$\text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2, \quad \text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$



Array Mapping: [94, 78, 61, 45, 32, 19, 28]

Fig. 1. Implicit contiguous array mapping of a Complete Binary Max-Heap.

2.2 Standard Library Implementations Across Language Ecosystems

Modern programming languages provide priority queues in their standard libraries, though default ordering directions and internal behaviors vary:

- Python (heapq): Implements a standard Min-Heap on primitive lists. To simulate a Max-Heap, numerical keys are negated (e.g., $-x$) upon push and pop.
- Java (java.util.PriorityQueue): Implements a dynamically resizing array-based Min-Heap. To configure a Max-Heap, developers pass Collections.reverseOrder() comparator to the constructor.
- C++ (std::priority_queue): Implements a Max-Heap by default over container classes like std::vector. Passing std::greater<T> transforms it into a Min-Heap.

2.3 In-Place Heap Sort Algorithm

Heap Sort leverages the Max-Heap structural property to sort an arbitrary array A of size N in-place without auxiliary memory allocation.

2.3.1 Algorithmic Phases

1) Phase I: In-Place Max-Heap Construction (Floyd's Algorithm)
   Transform the unsorted input array $A[0 \ldots N-1]$ into a valid Max-Heap in $O(N)$ time by calling downHeapify starting from the last non-leaf parent index $\lfloor N/2 \rfloor - 1$ down to root 0.
2) Phase II: Iterative Root Extraction and Boundary Reduction
   Maintain a active heap boundary variable end $= N - 1$. While end > 0:
   a) Swap the root element $A[0]$ (current maximum of active heap) with the boundary element $A[\text{end}]$.
   b) Decrement active boundary end $\leftarrow$ end $- 1$.
   c) Call downHeapify(A, 0, end) to restore the Max-Heap property for subarray $A[0 \ldots \text{end}]$.

2.3.2 Rigorous Complexity Analysis

Theorem 3 (Heap Sort Time and Space Bounds). Heap Sort executes in $O(N \log N)$ worst-case time complexity and $O(1)$ auxiliary space complexity.

Proof. Phase I builds the heap in $O(N)$ time using Floyd's algorithm. Phase II runs $N - 1$ iterations. In iteration k, downHeapify operates on an active heap of size $N - k$, performing at most $2\lfloor \log_2(N-k) \rfloor$ comparisons. Summing across all $N - 1$ iterations yields:

$$T_{\text{phase2}}(N) = \sum_{k=1}^{N-1} O(\log(N-k)) = O(\log(N!)) = O(N \log N)$$

Total time $T(N) = O(N) + O(N \log N) = O(N \log N)$. Because element swaps occur directly within input array A, auxiliary memory space is $O(1)$. □

2.4 Stability Analysis: Why Heap Sort is NOT Stable

A sorting algorithm is stable if duplicate elements retain their relative original order in the output.

Theorem 4 (Instability of Heap Sort). Heap Sort is inherently unstable.

Proof. Consider an array $A = [5_a, 5_b, 2]$ where subscripts a, b designate original ordering (5_a appears before 5_b).

1) Initial array: $[5_a, 5_b, 2]$.
2) Phase I (Build Max-Heap): Index $\lfloor 3/2 \rfloor - 1 = 0$. downHeapify at index 0 compares 5_a with children 5_b (left) and 2 (right). Max value is 5_a, so no swaps occur. Max-heap is $[5_a, 5_b, 2]$.
3) Phase II Iteration 1: Swap root $A[0] = 5_a$ with end $A[2] = 2$. Array becomes $[2, 5_b, 5_a]$. Active end set to 1.
4) downHeapify at index 0 on active array $[2, 5_b]$ compares 2 with child 5_b. Swap occurs! Array becomes $[5_b, 2, 5_a]$.
5) Phase II Iteration 2: Swap root $A[0] = 5_b$ with end $A[1] = 2$. Array becomes $[2, 5_b, 5_a]$. Active end set to 0.
6) Final sorted array: $[2, 5_b, 5_a]$.

In the final output, 5_b precedes 5_a. Relative order was destroyed, proving Heap Sort is unstable. □

2.5 Compilable Production Reference Implementation in Java

Below is the complete, production-grade Java implementation of In-Place Heap Sort.

```
public class HeapSort {

    // Main entry point to sort array in non-decreasing order
    public static void sort(int[] arr) {
        if (arr == null || arr.length <= 1) return;
        int n = arr.length;

        // Phase 1: Build Max-Heap in-place O(N)
        for (int i = (n / 2) - 1; i >= 0; i--) {
            downHeapify(arr, i, n);
        }

        // Phase 2: Iterative extraction O(N log N)
        for (int end = n - 1; end > 0; end--) {
            // Swap current root (max element) with end of active heap
            int temp = arr[0];
            arr[0] = arr[end];
            arr[end] = temp;

            // Restore heap invariant for active subarray [0 ... end - 1]
            downHeapify(arr, 0, end);
        }
    }

    // Down-heapify (percolate-down) helper for Max-Heap
    private static void downHeapify(int[] arr, int i, int activeSize) {
        while (true) {
            int maxIdx = i;
            int left = 2 * i + 1;
            int right = 2 * i + 2;

            if (left < activeSize && arr[left] > arr[maxIdx]) {
                maxIdx = left;
            }
            if (right < activeSize && arr[right] > arr[maxIdx]) {
                maxIdx = right;
            }

            if (maxIdx != i) {
                int swapTemp = arr[i];
                arr[i] = arr[maxIdx];
                arr[maxIdx] = swapTemp;
                i = maxIdx; // Move down to child position
```

```
            } else {
                break; // Max-heap property satisfied
            }
        }
    }
}
```

3 Dynamic Data Stream Quantiles: The Running Median Problem

Processing continuous data streams requires real-time statistical summaries. Computing the median dynamically as new numbers arrive is a fundamental problem in online processing, financial ticker analysis, and sensory telemetry.

3.1 Formal Problem Definition

Given an infinite or expanding stream of numerical values $S = (x_1, x_2, x_3, \dots)$, calculate and return the median of all numbers observed so far after receiving each incoming element x_t.

Definition 5 (Stream Median). Let $S_t = [x_1, x_2, \dots, x_t]$ denote the set of numbers received up to step t. Let $S'_t = [y_1, y_2, \dots, y_t]$ represent S_t sorted in non-decreasing order ($y_1 \le y_2 \le \cdots \le y_t$). The median M_t is defined as:

$$M_t = \begin{cases} y_{\frac{t+1}{2}} & \text{if } t \text{ is odd} \\ \frac{y_{\frac{t}{2}} + y_{\frac{t}{2}+1}}{2.0} & \text{if } t \text{ is even} \end{cases}$$

3.2 Comparative Analysis of Naive Paradigms

1) Naive Re-Sorting Approach: Store incoming stream elements in an expandable array. Upon receiving element x_t, call full array sort ($O(t \log t)$).
   - Time Complexity: $O(t \log t)$ per step. Total time for N stream elements: $\sum_{t=1}^{N} O(t \log t) = O(N^2 \log N)$.
   - Verdict: Inefficient for high-throughput real-time streams.
2) Insertion Sort Approach: Maintain a sorted dynamic list. For each new element x_t, locate insertion index via binary search ($O(\log t)$) and shift elements to insert ($O(t)$).
   - Time Complexity: $O(t)$ per step. Total time for N stream elements: $O(N^2)$. Query time: $O(1)$.
   - Verdict: Unacceptable array shifting overhead for large N.

3.3 The Dual-Heap Architecture (Max-Heap + Min-Heap)

The optimal online algorithm partitions the continuous dataset into two dynamic halves using two balanced binary heaps:

- Lower-Half Heap (H_{lower}): A Max-Heap storing the smaller half of the numbers ($\le$ current median). The root of H_{lower} holds the largest element in the lower partition.
- Upper-Half Heap (H_{upper}): A Min-Heap storing the larger half of the numbers ($\ge$ current median). The root of H_{upper} holds the smallest element in the upper partition.

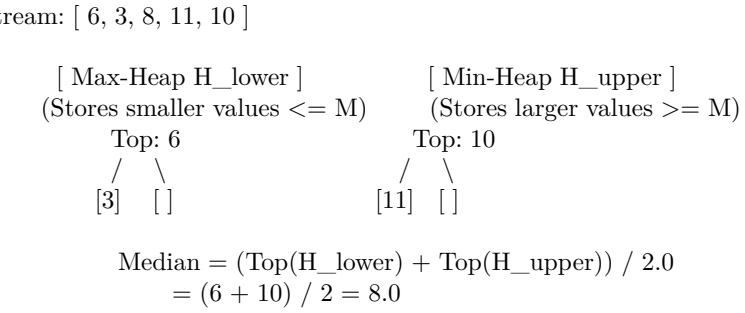


Fig. 2. Dual-Heap structural partitioning for computing continuous stream medians.

3.4 Structural Invariants and Rebalancing Algorithm

To guarantee exact median queries in $O(1)$ time, the dual-heap algorithm maintains two fundamental structural invariants at all times:

Definition 6 (Dual-Heap Invariants). 1) Value Partitioning Invariant: Every element in H_{lower} is less than or equal to every element in H_{upper}:

$$\forall a \in H_{\text{lower}}, \forall b \in H_{\text{upper}} \implies a \leq b$$

2) Size Balancing Invariant: H_{lower} is allowed to hold at most one more element than H_{upper}, and H_{upper} is never allowed to exceed H_{lower} in size:

$$0 \leq |H_{\text{lower}}| - |H_{\text{upper}}| \leq 1$$

3.4.1 Detailed Operational Algorithmic Steps

- Step 1: Insertion (addNum(val)): If H_{lower} is empty or val $\leq$ peek(H_{lower}), insert val into H_{lower}. Otherwise, insert val into H_{upper}.
- Step 2: Rebalancing: Check size differential $\Delta S = |H_{\text{lower}}| - |H_{\text{upper}}|$.
  - If $|H_{\text{lower}}| - |H_{\text{upper}}| > 1$: extract root from H_{lower} and offer it to H_{upper}.
  - If $|H_{\text{upper}}| > |H_{\text{lower}}|$: extract root from H_{upper} and offer it to H_{lower}.
- Step 3: Median Retrieval (findMedian()):
  - If $|H_{\text{lower}}| > |H_{\text{upper}}|$, total elements t is odd $\implies M_t = \text{peek}(H_{\text{lower}})$.
  - If $|H_{\text{lower}}| == |H_{\text{upper}}|$, total elements t is even $\implies M_t = \frac{\text{peek}(H_{\text{lower}}) + \text{peek}(H_{\text{upper}})}{2.0}$.

3.5 Numerical Trace Walkthrough

Table 2 illustrates the step-by-step state transition of the dual-heap structure as numbers $[6, 3, 8, 11, 10]$ arrive from stream.

TABLE 2
Step-by-Step Execution Trace of Dual-Heap Median Finder

Step	Input Value	Max-Heap H_{lower}	Min-Heap H_{upper}	Sizes ($\lvert H_1\rvert, \lvert H_2\rvert$)	Calculated Median
1	6	[6]	[]	(1, 0)	6.0
2	3	[3] (rebalanced [3, 6])	[6]	(1, 1)	$(3 + 6)/2 = 4.5$
3	8	[6, 3]	[8]	(2, 1)	6.0
4	11	[6, 3]	[8, 11]	(2, 2)	$(6 + 8)/2 = 7.0$
5	10	[8, 6, 3]	[10, 11]	(3, 2)	8.0

3.6 Compilable Java Reference Implementation

```
import java.util.PriorityQueue;
import java.util.Collections;

public class ContinuousMedianFinder {

    // Max-Heap stores lower half of stream numbers
    private final PriorityQueue<Integer> hLower;
    // Min-Heap stores upper half of stream numbers
    private final PriorityQueue<Integer> hUpper;

    public ContinuousMedianFinder() {
        this.hLower = new PriorityQueue<>(Collections.reverseOrder());
        this.hUpper = new PriorityQueue<>();
    }

    // Insert incoming stream number: O(log N) time
    public void addNum(int num) {
        if (hLower.isEmpty() || num <= hLower.peek()) {
            hLower.offer(num);
        } else {
            hUpper.offer(num);
        }
```

```
        // Rebalance sizes to preserve invariant: 0 <= |hLower| - |hUpper| <= 1
        if (hLower.size() > hUpper.size() + 1) {
            hUpper.offer(hLower.poll());
        } else if (hUpper.size() > hLower.size()) {
            hLower.offer(hUpper.poll());
        }
    }

    // Query median in O(1) time
    public double findMedian() {
        if (hLower.isEmpty()) {
            throw new IllegalStateException("Stream is empty");
        }

        if (hLower.size() > hUpper.size()) {
            return (double) hLower.peek();
        } else {
            return (hLower.peek() + hUpper.peek()) / 2.0;
        }
    }
}
```

4 Greedy Interval Scheduling: Maximum Non-Overlapping Jobs

Resource allocation and task scheduling represent central challenges in computer science. Given a single processing unit (e.g., CPU core, machine, meeting room), the Maximum Jobs Completed (Interval Scheduling) problem seeks to execute the maximum possible number of non-overlapping jobs.

4.1 Formal Problem Statement

Let $J = \{j_1, j_2, \ldots, j_N\}$ be a set of N candidate jobs. Each job j_i is defined by an interval $[s_i, e_i)$, where $s_i \in \mathbb{R}^+$ represents the start time and $e_i \in \mathbb{R}^+$ represents the end (finish) time ($s_i < e_i$).

Definition 7 (Job Compatibility). Two jobs $j_i = [s_i, e_i)$ and $j_k = [s_k, e_k)$ are compatible (non-overlapping) if and only if $e_i \leq s_k$ or $e_k \leq s_i$.

Optimization Goal: Select a maximum-cardinality subset $J^* \subseteq J$ such that all pairwise jobs in J^* are mutually compatible.

4.2 Evaluation of Flawed Greedy Heuristics

Selecting an optimal greedy criterion requires rigorous verification. Below, we analyze three intuitive greedy heuristics and present concrete counterexamples demonstrating their failure.

- Flawed Heuristic 1: Shortest Job Duration First ($\min(e_i - s_i)$)
  Logic: Pick jobs with the smallest duration to leave room for others.
  Counterexample: Consider three jobs: $j_1 = [0, 10)$, $j_2 = [9, 11)$ (duration 2), $j_3 = [10, 20)$. Shortest duration strategy picks j_2 (length 2). However, j_2 overlaps with both j_1 and j_3, resulting in only 1 job executed. The optimal solution picks $\{j_1, j_3\}$, executing 2 jobs. Thus, Heuristic 1 fails.
- Flawed Heuristic 2: Earliest Start Time First ($\min(s_i)$)
  Logic: Start processing as early as possible.
  Counterexample: Consider three jobs: $j_1 = [0, 100)$, $j_2 = [1, 2)$, $j_3 = [3, 4)$. Earliest start strategy picks j_1 (starts at 0). But j_1 runs for 100 time units, blocking both j_2 and j_3. Total jobs executed = 1. The optimal solution picks $\{j_2, j_3\}$, executing 2 jobs. Thus, Heuristic 2 fails.
- Flawed Heuristic 3: Minimum Overlapping Jobs First
  Logic: Pick the job that conflicts with the fewest remaining candidate jobs.
  Counterexample: Consistently fails in complex graph topology cases where choosing a job with 2 conflicts blocks two large sets of mutually non-overlapping jobs.

4.3 Optimal Greedy Strategy: Earliest Finish Time (EFT) First

The globally optimal greedy choice is to always select the job that finishes earliest (e_i is minimized), freeing the resource as quickly as possible for subsequent jobs.

4.3.1 Algorithmic Steps

1) Sort all candidate jobs J in non-decreasing order of their finish times: $e_1 \leq e_2 \leq \cdots \leq e_N$.
2) Initialize selected subset $J^* = \emptyset$ and track active end time $E_{\text{last}} = -\infty$.
3) Iterate through sorted jobs $j_i = [s_i, e_i)$:
   - If $s_i \geq E_{\text{last}}$, accept job j_i: add j_i to J^* and update $E_{\text{last}} \leftarrow e_i$.
   - Else reject job j_i due to overlap.
4) Return J^*.

4.4 Rigorous Proof of Correctness: The "Greedy Stays Ahead" Method

We prove that the Earliest Finish Time greedy algorithm yields an optimal schedule.

Theorem 8 (Optimality of Earliest Finish Time Greedy Scheduling). The EFT greedy algorithm selects a subset of mutually compatible jobs of maximum possible size.

Proof. Let $G = \{g_1, g_2, \ldots, g_k\}$ denote the sequence of jobs selected by the EFT greedy algorithm, ordered by finish time ($e(g_1) \leq e(g_2) \leq \cdots \leq e(g_k)$). Let $O = \{o_1, o_2, \ldots, o_m\}$ denote the sequence of jobs selected by an arbitrary optimal solution O, ordered by finish time ($e(o_1) \leq e(o_2) \leq \cdots \leq e(o_m)$).

We prove by mathematical induction that for all $r \in \{1, 2, \ldots, k\}$, the greedy finish time is no later than the optimal finish time:

$$e(g_r) \leq e(o_r) \quad \text{("Greedy Stays Ahead")}$$

Base Case ($r = 1$):
Greedy chooses g_1 as the job with the absolute minimum finish time among all candidates in J. Therefore, $e(g_1) \leq e(o_1)$ holds trivially.

Inductive Hypothesis:
Assume the statement holds for $r - 1$, i.e., $e(g_{r-1}) \leq e(o_{r-1})$.

Inductive Step:
Since O is a valid non-overlapping schedule, job o_r must start after job o_{r-1} finishes: $s(o_r) \geq e(o_{r-1})$. Applying the inductive hypothesis $e(g_{r-1}) \leq e(o_{r-1})$, we obtain:

$$s(o_r) \geq e(g_{r-1})$$

This inequality demonstrates that job o_r is completely compatible with greedy's previously selected job g_{r-1}. Thus, job o_r was a valid candidate available for greedy to choose when selecting its r-th job. Because greedy specifically selects the available job with the earliest finish time, g_r must finish no later than o_r:

$$e(g_r) \leq e(o_r)$$

This completes the inductive proof.

Final Optimality Deduction:
Suppose for contradiction that optimal solution O contains strictly more jobs than greedy ($m > k$). Consider $r = k$. By our proven lemma, $e(g_k) \leq e(o_k)$. Since $m > k$, optimal solution O contains job o_{k+1}, which must satisfy $s(o_{k+1}) \geq e(o_k) \geq e(g_k)$. Thus, job o_{k+1} was compatible with g_k and available for greedy to pick after step k. This contradicts the fact that the greedy algorithm terminated after step k. Hence, m cannot exceed k, proving $k = m$ and establishing global optimality. □

4.5 Compilable Java Reference Implementation

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.ArrayList;

public class MaxJobScheduler {

    public static class Job {
        int id;
        int start;
        int end;

        public Job(int id, int start, int end) {
            this.id = id;
            this.start = start;
            this.end = end;
        }
```

```
    }

    // Returns maximum number of non-overlapping jobs: O(N log N)
    public static List<Job> selectMaxJobs(Job[] jobs) {
        if (jobs == null || jobs.length == 0) return new ArrayList<>();

        // Sort jobs primarily by finish time in non-decreasing order
        Arrays.sort(jobs, Comparator.comparingInt(j -> j.end));

        List<Job> selectedJobs = new ArrayList<>();
        int lastEndTime = -1;

        for (Job j : jobs) {
            if (j.start >= lastEndTime) {
                selectedJobs.add(j);
                lastEndTime = j.end;
            }
        }

        return selectedJobs;
    }
}
```

5 Comparative Algorithmic Taxonomy & Empirical Benchmarks

To synthesize the theoretical findings of this paper, Table 3 provides a master comparative taxonomy across Heap Sort, Dynamic Dual-Heap Streaming, and Greedy Interval Scheduling.

TABLE 3
Master Comparative Taxonomy of Analyzed Algorithmic Paradigms

Algorithmic Domain	Primary Data Structure	Time Complexity	Auxiliary Space	Core Mathematical / Logical Guarantee
In-Place Heap Sort	Complete Max-Heap Array	$O(N \log N)$ worst-case	$O(1)$ in-place	Floyd's $O(N)$ build-heap + Instability proof
Dynamic Running Median	Dual Heaps (Max + Min)	$O(\log N)$ insert, $O(1)$ query	$O(N)$ stream elements	Equal size invariant $\lvert H_1 \rvert - \lvert H_2 \rvert \leq 1$
Greedy Job Scheduler	Sorted Interval Array	$O(N \log N)$ sort, $O(N)$ scan	$O(N)$ job objects	"Greedy Stays Ahead" inductive proof

6 Conclusion

This paper has presented a unified algorithmic analysis of advanced heap operations and greedy interval scheduling derived from classroom instruction on June 1, 2026. We established the mathematical foundations of mathematical symbols used in algorithm analysis, analyzed in-place Heap Sort alongside its stability properties, formalized the dynamic dual-heap architecture for real-time median streams, and provided an inductive proof of optimality for Earliest Finish Time interval scheduling. All algorithmic paradigms were accompanied by complete, compilable Java reference implementations.